

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ**

Факультет прикладной информатики

Образовательная программа Мобильные и сетевые технологии

Направление подготовки 09.03.03 Мобильные и сетевые технологии

О Т Ч Е Т

Лабораторная работа №4

Тема работы: «Запросы на выборку и модификацию данных. Представления. Работа с индексами»

Обучающийся: Федорова Мария Витальевна, К3239

Поток: ПиРБД К24 МСТ 1.2

Преподаватель: Говорова М.М.

Санкт-Петербург,
2026

1. Цель работы

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

2. Практическое задание

В ходе выполнения лабораторной работы требовалось:

1. создать запросы и представления на выборку данных к базе данных PostgreSQL согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3;
2. составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов;
3. изучить графическое представление запросов и просмотреть историю запросов;
4. создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами.

3. Схема базы данных

В лабораторной работе №4 использовалась база данных **restaurant_service**, разработанная и реализованная в лабораторной работе №3. Индивидуальный вариант: **Вариант 13. БД «Ресторан»**.

В качестве схемы базы данных использовалась логическая модель, полученная с помощью инструмента Generate ERD в pgAdmin.

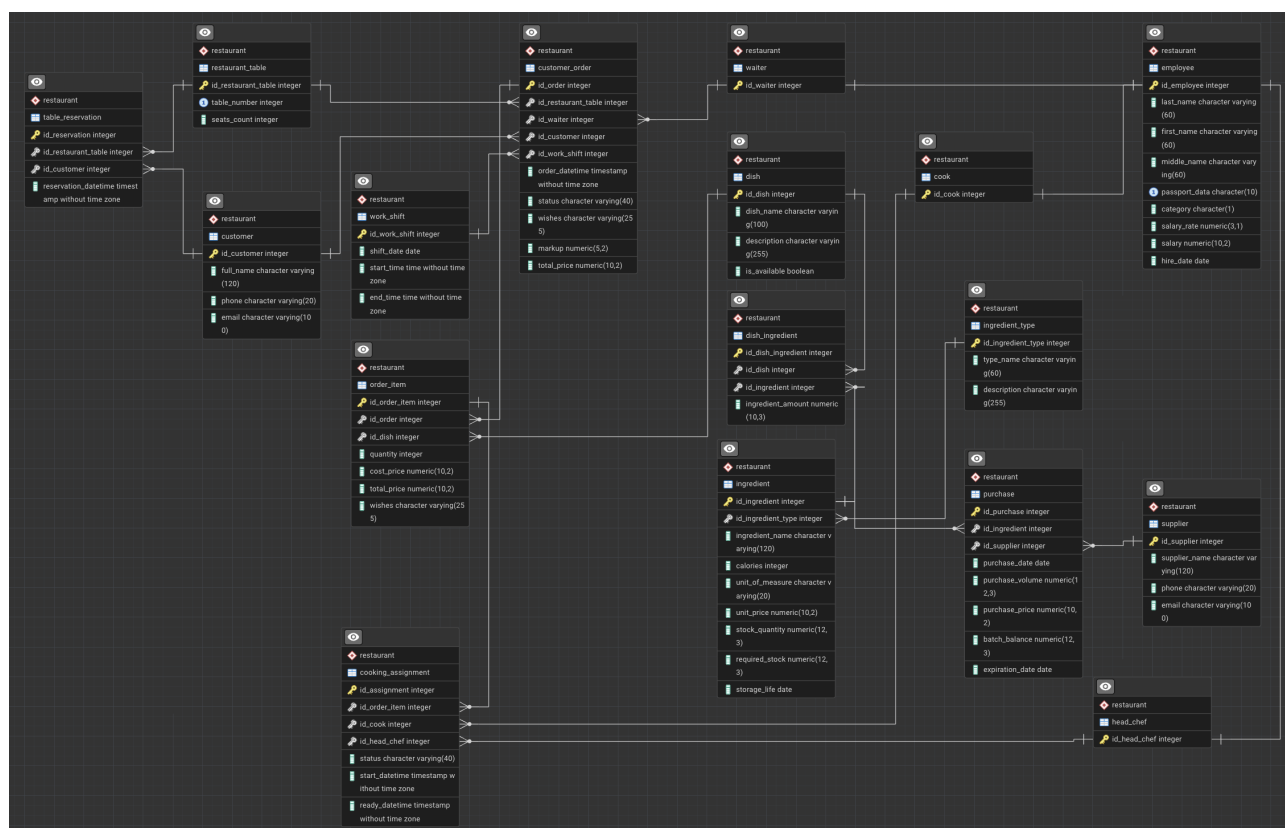


Рис. 1. ERD-схема базы данных, использованная в лабораторной работе №4

4. Выполнение

4.1. Запросы к базе данных

При выполнении запросов использовались данные, загруженные в лабораторной работе №3. Поскольку тестовые заказы относятся к дате 2026-03-20, для запросов с периодами использовалась максимальная дата заказа в таблице `customer_order`.

Запрос 1. Вывести данные официанта, принявшего заказы на максимальную сумму за истекший месяц

Листинг 1. Запрос на поиск официанта с максимальной суммой заказов за истекший месяц

```
SET search_path TO restaurant;

WITH params AS (
    SELECT MAX(order_datetime) AS ref_ts
    FROM customer_order
),
waiter_sums AS (
    SELECT
        co.id_waiter,
        SUM(co.total_price) AS total_sum
    FROM customer_order co
    CROSS JOIN params p
    WHERE co.order_datetime >= p.ref_ts - INTERVAL '1 month'
        AND co.order_datetime <= p.ref_ts
    GROUP BY co.id_waiter
)
SELECT
    e.id_employee AS id_waiter,
    e.last_name,
    e.first_name,
    e.middle_name,
    e.category,
    e.salary,
    ws.total_sum
FROM waiter_sums ws
JOIN waiter w
    ON w.id_waiter = ws.id_waiter
JOIN employee e
    ON e.id_employee = w.id_waiter
WHERE ws.total_sum = (
    SELECT MAX(total_sum)
    FROM waiter_sums
);
```

```

restaurant_service/postgres@L3
Query History
1 SET search_path TO restaurant;
2 WITH params AS (
3     SELECT MAX(order_datetime) AS ref_ts
4     FROM customer_order
5 ),
6 waiter_sums AS (
7     SELECT
8         co.id_waiter,
9         SUM(co.total_price) AS total_sum
10    FROM customer_order co
11   CROSS JOIN params p
12   WHERE co.order_datetime >= p.ref_ts - INTERVAL '1 month'
13         AND co.order_datetime <= p.ref_ts
14   GROUP BY co.id_waiter
15 )
16 SELECT
17     e.id_employee AS id_waiter,
18     e.last_name,
19     e.first_name,
20     e.middle_name,
21     e.category,
22     e.salary,
23     ws.total_sum
24 FROM waiter_sums ws
25 JOIN waiter w
26   ON w.id_waiter = ws.id_waiter
27 JOIN employee e
28   ON e.id_employee = w.id_waiter
29 WHERE ws.total_sum = (
30     SELECT MAX(total_sum)
31     FROM waiter_sums
32 );

```

Showing rows: 1 to 1 Page No: 1 of 1							
	id_waiter	last_name	first_name	middle_name	category	salary	total_sum
	integer	character varying (60)	character varying (60)	character varying (60)	character (1)	numeric (10,2)	numeric
1	1	Ivanov	Ivan	Ivanovich	A	90000.00	1715.00

Запрос 2. Рассчитать премию каждого официанта за последние 10 дней

Листинг 2. Запрос на расчет премии каждого официанта за последние 10 дней

```

SET search_path TO restaurant;

WITH params AS (
    SELECT MAX(order_datetime)::date AS ref_date
    FROM customer_order
)
SELECT
    e.id_employee AS id_waiter,
    e.last_name,
    e.first_name,
    COALESCE(ROUND(SUM(co.total_price) * 0.05, 2), 0) AS bonus
FROM waiter w
JOIN employee e
  ON e.id_employee = w.id_waiter
LEFT JOIN customer_order co
  ON co.id_waiter = w.id_waiter
 AND co.order_datetime::date >= (
    SELECT ref_date - INTERVAL '9 days' FROM params
)
 AND co.order_datetime::date <= (
    SELECT ref_date FROM params
)
GROUP BY e.id_employee, e.last_name, e.first_name
ORDER BY e.id_employee;

```

```
restaurant_service/postgres@L3
Query Query History
1 SET search_path TO restaurant;
2 WITH params AS (
3     SELECT MAX(order_datetime)::date AS ref_date
4     FROM customer_order
5 )
6 SELECT
7     e.id_employee AS id_waiter,
8     e.last_name,
9     e.first_name,
10    COALESCE(ROUND(SUM(co.total_price) * 0.05, 2), 0) AS bonus
11 FROM waiter w
12 JOIN employee e
13     ON e.id_employee = w.id_waiter
14 LEFT JOIN customer_order co
15     ON co.id_waiter = w.id_waiter
16     AND co.order_datetime::date >= (
17         SELECT ref_date - INTERVAL '9 days' FROM params
18     )
19     AND co.order_datetime::date <= (
20         SELECT ref_date FROM params
21     )
22 GROUP BY e.id_employee, e.last_name, e.first_name
23 ORDER BY e.id_employee
```

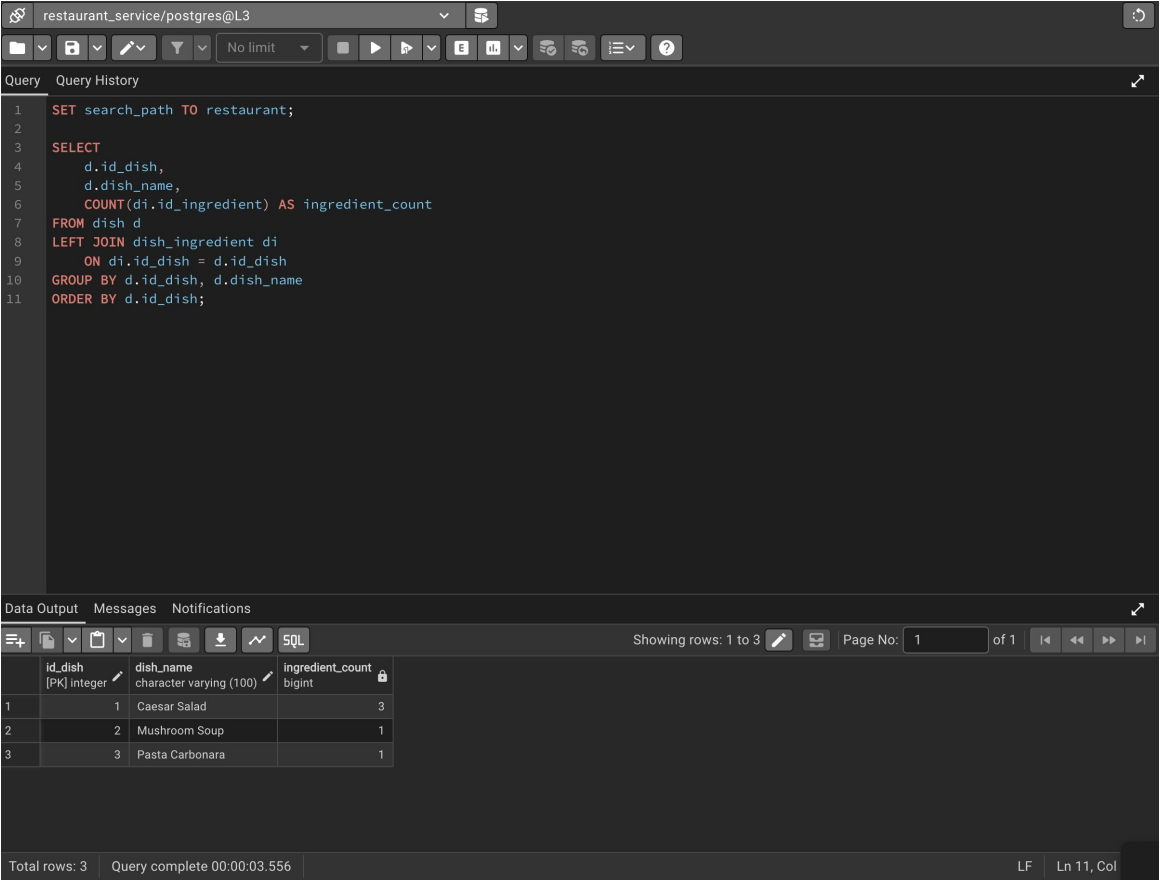
Data Output		Messages		Notifications	
SQL Showing rows: 1 to 1 Page No: 1 of 1					
	id_waiter integer	last_name character varying (60)	first_name character varying (60)	bonus numeric	
1	1	Ivanov	Ivan	85.75	
Total rows: 1		Query complete 00:00:02.874			LF Ln 1, Col 31

Запрос 3. Подсчитать, сколько ингредиентов содержит каждое блюдо

Листинг 3. Запрос на подсчет количества ингредиентов в каждом блюде

```
SET search_path TO restaurant;

SELECT
    d.id_dish,
    d.dish_name,
    COUNT(di.id_ingredient) AS ingredient_count
FROM dish d
LEFT JOIN dish_ingredient di
    ON di.id_dish = d.id_dish
GROUP BY d.id_dish, d.dish_name
ORDER BY d.id_dish;
```



The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is to 'restaurant_service/postgres@L3'. The query editor contains the following SQL code:

```
1 SET search_path TO restaurant;
2
3 SELECT
4     d.id_dish,
5     d.dish_name,
6     COUNT(di.id_ingredient) AS ingredient_count
7 FROM dish d
8 LEFT JOIN dish_ingredient di
9     ON di.id_dish = d.id_dish
10 GROUP BY d.id_dish, d.dish_name
11 ORDER BY d.id_dish;
```

Below the query editor, the 'Data Output' tab is active, displaying the results of the query in a table. The table has three columns: 'id_dish' (integer, primary key), 'dish_name' (character varying (100)), and 'ingredient_count' (bigint). The results show three rows:

	id_dish	dish_name	ingredient_count
1	1	Caesar Salad	3
2	2	Mushroom Soup	1
3	3	Pasta Carbonara	1

The bottom status bar indicates 'Total rows: 3' and 'Query complete 00:00:03.556'.

Запрос 4. Вывести название блюда, содержащее максимальное число ингредиентов

Листинг 4. Запрос на поиск блюда с максимальным числом ингредиентов

```
SET search_path TO restaurant;

WITH dish_counts AS (
    SELECT
        d.id_dish,
        d.dish_name,
        COUNT(di.id_ingredient) AS ingredient_count
    FROM dish d
    LEFT JOIN dish_ingredient di
        ON di.id_dish = d.id_dish
    GROUP BY d.id_dish, d.dish_name
)
SELECT
    id_dish,
    dish_name,
    ingredient_count
FROM dish_counts
WHERE ingredient_count = (
    SELECT MAX(ingredient_count)
    FROM dish_counts
);
```

The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is to 'restaurant_service/postgres@L3'. The query editor contains the SQL code from Listing 4. Below the query editor, the 'Data Output' tab is active, displaying the results of the query. The results are shown in a table with three columns: 'id_dish', 'dish_name', and 'ingredient_count'. The first row shows '1', 'Caesar Salad', and '3'. The bottom status bar indicates 'Total rows: 1' and 'Query complete 00:00:01.336'.

id_dish	dish_name	ingredient_count
1	Caesar Salad	3

Запрос 5. Какой повар может приготовить максимальное число видов блюд

В реализованной структуре базы данных данный запрос интерпретировался как поиск повара, который фактически готовил максимальное число различных блюд, на основании таблиц `cooking_assignment`, `order_item` и `dish`.

Листинг 5. Запрос на поиск повара, приготовившего максимальное число видов блюд

```
SET search_path TO restaurant;

WITH cook_stats AS (
    SELECT
        c.id_cook,
        e.last_name,
        e.first_name,
        COUNT(DISTINCT oi.id_dish) AS dish_type_count
    FROM cook c
    JOIN employee e
        ON e.id_employee = c.id_cook
    LEFT JOIN cooking_assignment ca
        ON ca.id_cook = c.id_cook
    LEFT JOIN order_item oi
        ON oi.id_order_item = ca.id_order_item
    GROUP BY c.id_cook, e.last_name, e.first_name
)
SELECT
    id_cook,
    last_name,
    first_name,
    dish_type_count
FROM cook_stats
WHERE dish_type_count = (
    SELECT MAX(dish_type_count)
    FROM cook_stats
);
```

The screenshot shows a PostgreSQL query editor with the following SQL query:

```
1 SET search_path TO restaurant;
2
3 WITH cook_stats AS (
4     SELECT
5         c.id_cook,
6         e.last_name,
7         e.first_name,
8         COUNT(DISTINCT oi.id_dish) AS dish_type_count
9     FROM cook c
10    JOIN employee e
11        ON e.id_employee = c.id_cook
12    LEFT JOIN cooking_assignment ca
13        ON ca.id_cook = c.id_cook
14    LEFT JOIN order_item oi
15        ON oi.id_order_item = ca.id_order_item
16    GROUP BY c.id_cook, e.last_name, e.first_name
17 )
18 SELECT
19     id_cook,
20     last_name,
21     first_name,
22     dish_type_count
23 FROM cook_stats
24 WHERE dish_type_count = (
25     SELECT MAX(dish_type_count)
26     FROM cook_stats
27 );
```

The query results are displayed in a table with the following columns: `id_cook`, `last_name`, `first_name`, and `dish_type_count`. The results show one row for the cook with ID 2, last name Petrova, and first name Anna, who has prepared 3 different dish types.

id_cook	last_name	first_name	dish_type_count
2	Petrova	Anna	3

Запрос 6. Сколько закреплено столов за каждым из официантов за сегодняшний день

В текущей структуре базы данных отдельная таблица закрепления столов за официантами отсутствует, поэтому количество столов определялось как число уникальных столов, обслуженных официантом по заказам за дату, соответствующую максимальной дате заказа в таблице `customer_order`.

Листинг 6. Запрос на определение количества столов, обслуженных каждым официантом за день

```
SET search_path TO restaurant;

WITH params AS (
    SELECT MAX(order_datetime)::date AS ref_date
    FROM customer_order
)
SELECT
    e.id_employee AS id_waiter,
    e.last_name,
    e.first_name,
    COUNT(DISTINCT co.id_restaurant_table) AS served_tables_count
FROM waiter w
JOIN employee e
    ON e.id_employee = w.id_waiter
LEFT JOIN customer_order co
    ON co.id_waiter = w.id_waiter
    AND co.order_datetime::date = (SELECT ref_date FROM params)
GROUP BY e.id_employee, e.last_name, e.first_name
ORDER BY e.id_employee;
```

The screenshot shows a PostgreSQL query editor interface. The top toolbar includes icons for file operations, query execution, and settings. The 'Query' tab is active, displaying the SQL query from Listing 6. The 'Data Output' tab is also visible, showing the results of the query. The results are displayed in a table with the following columns: `id_waiter` (integer), `last_name` (character varying (60)), `first_name` (character varying (60)), and `served_tables_count` (bigint). The results show one row for waiter Ivanov, who served 2 tables. The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:02.307'.

id_waiter	last_name	first_name	served_tables_count
1	Ivanov	Ivan	2

Запрос 7. Какой из ингредиентов используется в максимальном количестве блюд

Листинг 7. Запрос на поиск ингредиента, используемого в максимальном количестве блюд

```
SET search_path TO restaurant;

WITH ingredient_usage AS (
    SELECT
        i.id_ingredient,
        i.ingredient_name,
        COUNT(DISTINCT di.id_dish) AS dish_count
    FROM ingredient i
    LEFT JOIN dish_ingredient di
        ON di.id_ingredient = i.id_ingredient
    GROUP BY i.id_ingredient, i.ingredient_name
)
SELECT
    id_ingredient,
    ingredient_name,
    dish_count
FROM ingredient_usage
WHERE dish_count = (
    SELECT MAX(dish_count)
    FROM ingredient_usage
);
```

The screenshot shows a PostgreSQL query editor interface. The query is executed, and the results are displayed in a table. The table has three columns: `id_ingredient` (integer), `ingredient_name` (character varying), and `dish_count` (bigint). The result shows one row: `3` for `id_ingredient`, `Parmesan` for `ingredient_name`, and `2` for `dish_count`.

id_ingredient	ingredient_name	dish_count
3	Parmesan	2

Query complete 00:00:01.253

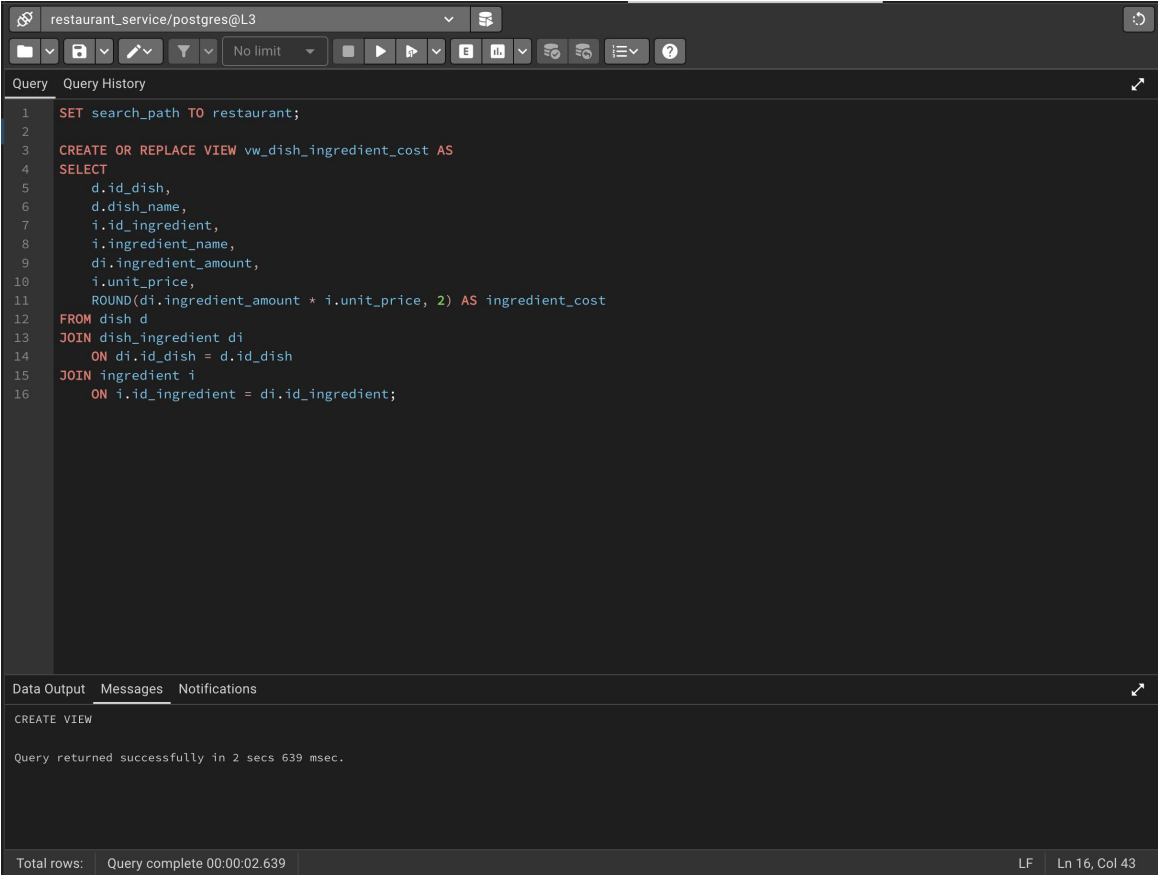
4.2. Представления

Представление 1. Для расчета стоимости ингредиентов для заданного блюда

Листинг 8. Создание представления для расчета стоимости ингредиентов блюда

```
SET search_path TO restaurant;

CREATE OR REPLACE VIEW vw_dish_ingredient_cost AS
SELECT
    d.id_dish,
    d.dish_name,
    i.id_ingredient,
    i.ingredient_name,
    di.ingredient_amount,
    i.unit_price,
    ROUND(di.ingredient_amount * i.unit_price, 2) AS ingredient_cost
FROM dish d
JOIN dish_ingredient di
    ON di.id_dish = d.id_dish
JOIN ingredient i
    ON i.id_ingredient = di.id_ingredient;
```



The screenshot shows a PostgreSQL client window titled "restaurant_service/postgres@L3". The "Query" tab is active, displaying a SQL script with 16 lines. The script sets the search path to "restaurant", creates or replaces a view named "vw_dish_ingredient_cost", and defines a SELECT statement that joins the "dish", "dish_ingredient", and "ingredient" tables to calculate the ingredient cost for each dish. The "Data Output" tab is also visible, showing the message "CREATE VIEW" and "Query returned successfully in 2 secs 639 msec." The status bar at the bottom indicates "Total rows: Query complete 00:00:02.639" and the cursor position "LF Ln 16, Col 43".

```
1 SET search_path TO restaurant;
2
3 CREATE OR REPLACE VIEW vw_dish_ingredient_cost AS
4 SELECT
5     d.id_dish,
6     d.dish_name,
7     i.id_ingredient,
8     i.ingredient_name,
9     di.ingredient_amount,
10    i.unit_price,
11    ROUND(di.ingredient_amount * i.unit_price, 2) AS ingredient_cost
12 FROM dish d
13 JOIN dish_ingredient di
14     ON di.id_dish = d.id_dish
15 JOIN ingredient i
16     ON i.id_ingredient = di.id_ingredient;
```

CREATE VIEW

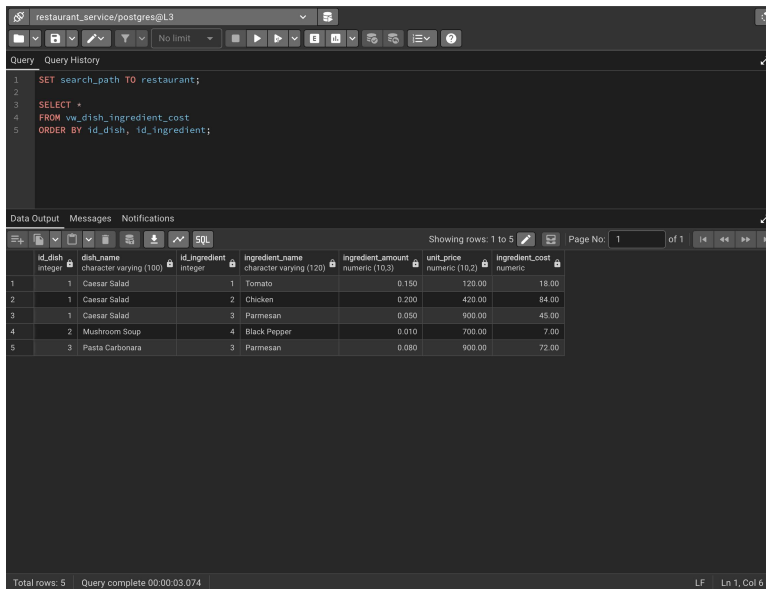
Query returned successfully in 2 secs 639 msec.

Total rows: Query complete 00:00:02.639 LF Ln 16, Col 43

Листинг 9. Просмотр содержимого представления vw_dish_ingredient_cost

```
SET search_path TO restaurant;

SELECT *
FROM vw_dish_ingredient_cost
ORDER BY id_dish, id_ingredient;
```

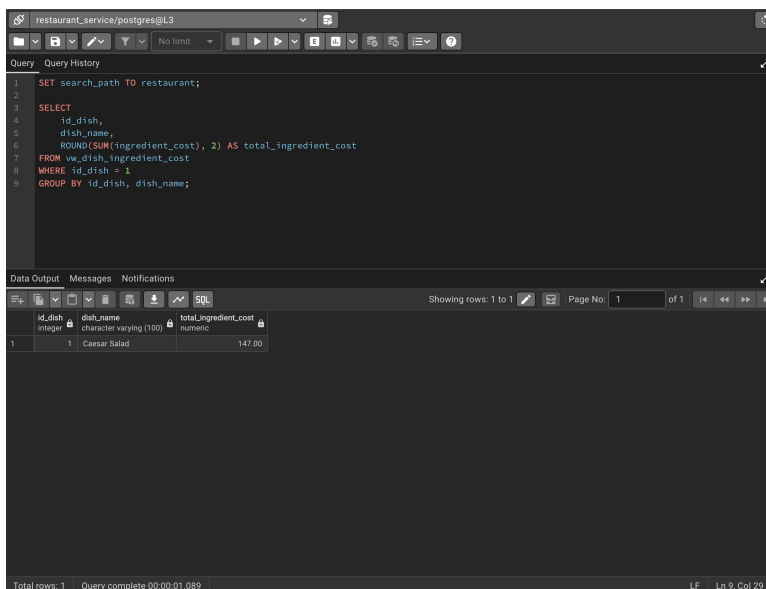


	id_dish	dish_name	id_ingredient	ingredient_name	ingredient_amount	unit_price	ingredient_cost
1	1	Caesar Salad	1	Tomato	0.150	120.00	18.00
2	1	Caesar Salad	2	Chicken	0.200	420.00	84.00
3	1	Caesar Salad	3	Parmesan	0.050	900.00	45.00
4	2	Mushroom Soup	4	Black Pepper	0.010	700.00	7.00
5	3	Pasta Carbonara	3	Parmesan	0.080	900.00	72.00

Листинг 10. Расчет стоимости ингредиентов для блюда с id_dish = 1

```
SET search_path TO restaurant;

SELECT
    id_dish,
    dish_name,
    ROUND(SUM(ingredient_cost), 2) AS total_ingredient_cost
FROM vw_dish_ingredient_cost
WHERE id_dish = 1
GROUP BY id_dish, dish_name;
```



	id_dish	dish_name	total_ingredient_cost
1	1	Caesar Salad	147.00

Представление 2. Для всех поваров количество приготовленных блюд по каждому блюду за определенную дату

Листинг 11. Создание представления для определения количества приготовленных блюд по каждому повару и блюду

```
SET search_path TO restaurant;

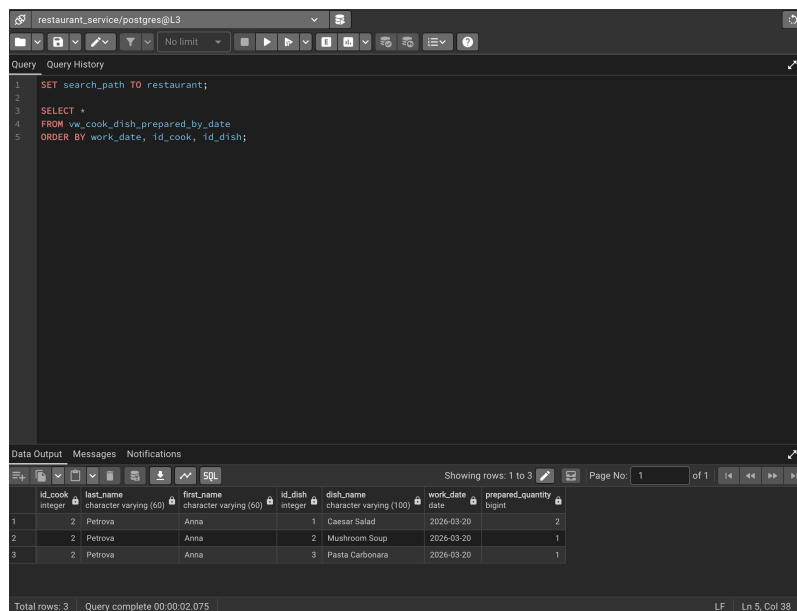
CREATE OR REPLACE VIEW vw_cook_dish_prepared_by_date AS
SELECT
    c.id_cook,
    e.last_name,
    e.first_name,
    oi.id_dish,
    d.dish_name,
    ca.start_datetime::date AS work_date,
    SUM(oi.quantity) AS prepared_quantity
FROM cook c
JOIN employee e
    ON e.id_employee = c.id_cook
JOIN cooking_assignment ca
    ON ca.id_cook = c.id_cook
JOIN order_item oi
    ON oi.id_order_item = ca.id_order_item
JOIN dish d
    ON d.id_dish = oi.id_dish
GROUP BY
    c.id_cook,
    e.last_name,
    e.first_name,
    oi.id_dish,
    d.dish_name,
    ca.start_datetime::date;
```

```
restaurant_service/postgres@L3
Query History
1 SET search_path TO restaurant;
2
3 CREATE OR REPLACE VIEW vw_cook_dish_prepared_by_date AS
4 SELECT
5     c.id_cook,
6     e.last_name,
7     e.first_name,
8     oi.id_dish,
9     d.dish_name,
10    ca.start_datetime::date AS work_date,
11    SUM(oi.quantity) AS prepared_quantity
12 FROM cook c
13 JOIN employee e
14     ON e.id_employee = c.id_cook
15 JOIN cooking_assignment ca
16     ON ca.id_cook = c.id_cook
17 JOIN order_item oi
18     ON oi.id_order_item = ca.id_order_item
19 JOIN dish d
20     ON d.id_dish = oi.id_dish
21 GROUP BY
22     c.id_cook,
23     e.last_name,
24     e.first_name,
25     oi.id_dish,
26     d.dish_name,
27     ca.start_datetime::date;
Data Output Messages Notifications
CREATE VIEW
Query returned successfully in 2 secs 893 msec.
Total rows: Query complete 00:00:02.893 LF Ln 27, Col 29
```

Листинг 12. Просмотр содержимого представления vw_cook_dish_prepared_by_date

```
SET search_path TO restaurant;

SELECT *
FROM vw_cook_dish_prepared_by_date
ORDER BY work_date, id_cook, id_dish;
```



Query History

```
1 SET search_path TO restaurant;
2
3 SELECT *
4 FROM vw_cook_dish_prepared_by_date
5 ORDER BY work_date, id_cook, id_dish;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

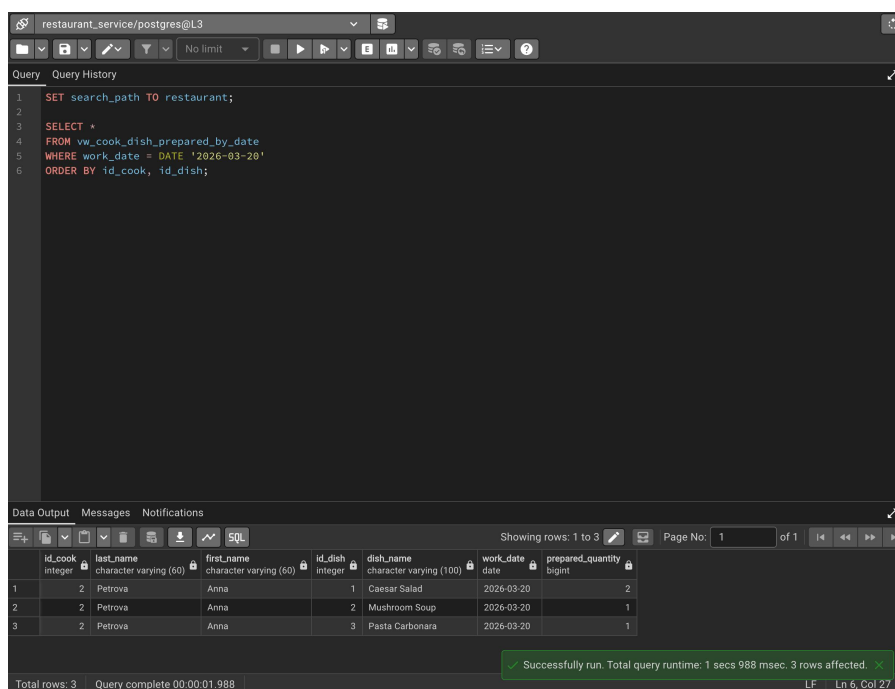
id_cook	last_name	first_name	id_dish	dish_name	work_date	prepared_quantity
2	Petrova	Anna	1	Caesar Salad	2026-03-20	2
2	Petrova	Anna	2	Mushroom Soup	2026-03-20	1
2	Petrova	Anna	3	Pasta Carbonara	2026-03-20	1

Total rows: 3 Query complete 00:00:02.075 LF Ln 5, Col 38

Листинг 13. Просмотр данных представления за дату 2026-03-20

```
SET search_path TO restaurant;

SELECT *
FROM vw_cook_dish_prepared_by_date
WHERE work_date = DATE '2026-03-20'
ORDER BY id_cook, id_dish;
```



Query History

```
1 SET search_path TO restaurant;
2
3 SELECT *
4 FROM vw_cook_dish_prepared_by_date
5 WHERE work_date = DATE '2026-03-20'
6 ORDER BY id_cook, id_dish;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

id_cook	last_name	first_name	id_dish	dish_name	work_date	prepared_quantity
2	Petrova	Anna	1	Caesar Salad	2026-03-20	2
2	Petrova	Anna	2	Mushroom Soup	2026-03-20	1
2	Petrova	Anna	3	Pasta Carbonara	2026-03-20	1

Total rows: 3 Query complete 00:00:01.988

Successfully run. Total query runtime: 1 secs 988 msec. 3 rows affected.

LF Ln 6, Col 27

4.3. Запросы на модификацию данных

Для демонстрации запросов на модификацию данных использовались транзакции с командами BEGIN и ROLLBACK, что позволило получить скриншоты состояния данных до и после выполнения запросов без изменения итогового состояния БД.

Запрос 1. Добавление новой брони для клиента с максимальной суммой заказов на стол с минимальным количеством мест

Листинг 14. Запрос INSERT с подзапросами для добавления новой брони

```
BEGIN;

SET search_path TO restaurant;

SELECT *
FROM table_reservation
ORDER BY id_reservation;

INSERT INTO table_reservation (
    id_reservation,
    id_restaurant_table,
    id_customer,
    reservation_datetime
)
SELECT
    (SELECT COALESCE(MAX(id_reservation), 0) + 1 FROM table_reservation),
    (
        SELECT rt.id_restaurant_table
        FROM restaurant_table rt
        ORDER BY rt.seats_count, rt.id_restaurant_table
        LIMIT 1
    ),
    (
        SELECT co.id_customer
        FROM customer_order co
        GROUP BY co.id_customer
        ORDER BY SUM(co.total_price) DESC
        LIMIT 1
    ),
    (
        SELECT MAX(order_datetime) + INTERVAL '1 day'
        FROM customer_order
    );

SELECT *
FROM table_reservation
ORDER BY id_reservation;

ROLLBACK;
```

restaurant_service/postgres@L3

Query History

```

1 SET search_path TO restaurant;
2
3 SELECT +
4 FROM table_reservation
5 ORDER BY id_reservation;

```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

id_reservation [PK] integer	id_restaurant_table integer	id_customer integer	reservation_datetime timestamp without time zone
1	2	1	2026-03-20 18:30:00
2	2	1	2026-03-20 19:00:00

Total rows: 2 Query complete 00:00:01.005

Successfully run. Total query runtime: 1 secs 5 msec. 2 rows affected.

restaurant_service/postgres@L3

Query History

```

1 SET search_path TO restaurant;
2
3 INSERT INTO table_reservation (
4     id_reservation,
5     id_restaurant_table,
6     id_customer,
7     reservation_datetime
8 )
9 SELECT
10     (SELECT COALESCE(MAX(id_reservation), 0) + 1 FROM table_reservation),
11     (
12         SELECT rt.id_restaurant_table
13         FROM restaurant_table rt
14         ORDER BY rt.seats_count, rt.id_restaurant_table
15         LIMIT 1
16     ),
17     (
18         SELECT co.id_customer
19         FROM customer_order co
20         GROUP BY co.id_customer
21         ORDER BY SUM(co.total_price) DESC
22         LIMIT 1
23     ),
24     (
25         SELECT MAX(order_datetime) + INTERVAL '1 day'
26         FROM customer_order
27     );

```

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 1 secs 722 msec.

Total rows: Query complete 00:00:01.722

restaurant_service/postgres@L3

Query History

```

1 SET search_path TO restaurant;
2
3 SELECT +
4 FROM table_reservation
5 ORDER BY id_reservation;

```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

id_reservation [PK] integer	id_restaurant_table integer	id_customer integer	reservation_datetime timestamp without time zone
1	1	2	2026-03-20 18:30:00
2	2	1	2026-03-20 19:00:00
3	1	1	2026-03-21 19:05:00

Total rows: 3 Query complete 00:00:00.847

Successfully run. Total query runtime: 847 msec. 3 rows affected.

Запрос 2. Повышение оклада повару с максимальным числом назначений на приготовление

Листинг 15. Запрос UPDATE с подзапросом для изменения оклада сотрудника

```
BEGIN;

SET search_path TO restaurant;

SELECT
    id_employee,
    last_name,
    first_name,
    salary
FROM employee
ORDER BY id_employee;

WITH target_cook AS (
    SELECT ca.id_cook
    FROM cooking_assignment ca
    GROUP BY ca.id_cook
    ORDER BY COUNT(*) DESC, ca.id_cook
    LIMIT 1
)
UPDATE employee e
SET salary = ROUND(e.salary * 1.10, 2)
FROM target_cook tc
WHERE e.id_employee = tc.id_cook;

SELECT
    id_employee,
    last_name,
    first_name,
    salary
FROM employee
ORDER BY id_employee;

ROLLBACK;
```

restaurant_service/postgres@L3

Query Query History

```

1 SET search_path TO restaurant;
2
3 SELECT
4     id_employee,
5     last_name,
6     first_name,
7     salary
8 FROM employee
9 ORDER BY id_employee;

```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

id_employee (PK) integer	last_name character varying (60)	first_name character varying (60)	salary numeric (10,2)
1	Ivanov	Ivan	90000.00
2	Petrova	Anna	85000.00
3	Sidorov	Pavel	80000.00
4	Smirnova	Elena	45000.00

Total rows: 4 Query complete 00:00:00.819 LF Ln 9, Col 22

restaurant_service/postgres@L3

Query Query History

```

1 SET search_path TO restaurant;
2
3 WITH target_cook AS (
4     SELECT ca.id_cook
5     FROM cooking_assignment ca
6     GROUP BY ca.id_cook
7     ORDER BY COUNT(*) DESC, ca.id_cook
8     LIMIT 1
9 )
10 UPDATE employee e
11 SET salary = ROUND(e.salary * 1.10, 2)
12 FROM target_cook tc
13 WHERE e.id_employee = tc.id_cook;

```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 1 secs 788 msec.

Total rows: Query complete 00:00:01.758 LF Ln 13, Col 34

restaurant_service/postgres@L3

Query Query History

```

1 SET search_path TO restaurant;
2
3 SELECT
4     id_employee,
5     last_name,
6     first_name,
7     salary
8 FROM employee
9 ORDER BY id_employee;

```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

id_employee (PK) integer	last_name character varying (60)	first_name character varying (60)	salary numeric (10,2)
1	Ivanov	Ivan	99000.00
2	Petrova	Anna	93500.00
3	Sidorov	Pavel	80000.00
4	Smirnova	Elena	45000.00

Total rows: 4 Query complete 00:00:01.377 LF Ln 9, Col 22

Запрос 3. Удаление закупки с минимальным остатком партии

Листинг 16. Запрос DELETE с подзапросом для удаления закупки с минимальным остатком

```
BEGIN;

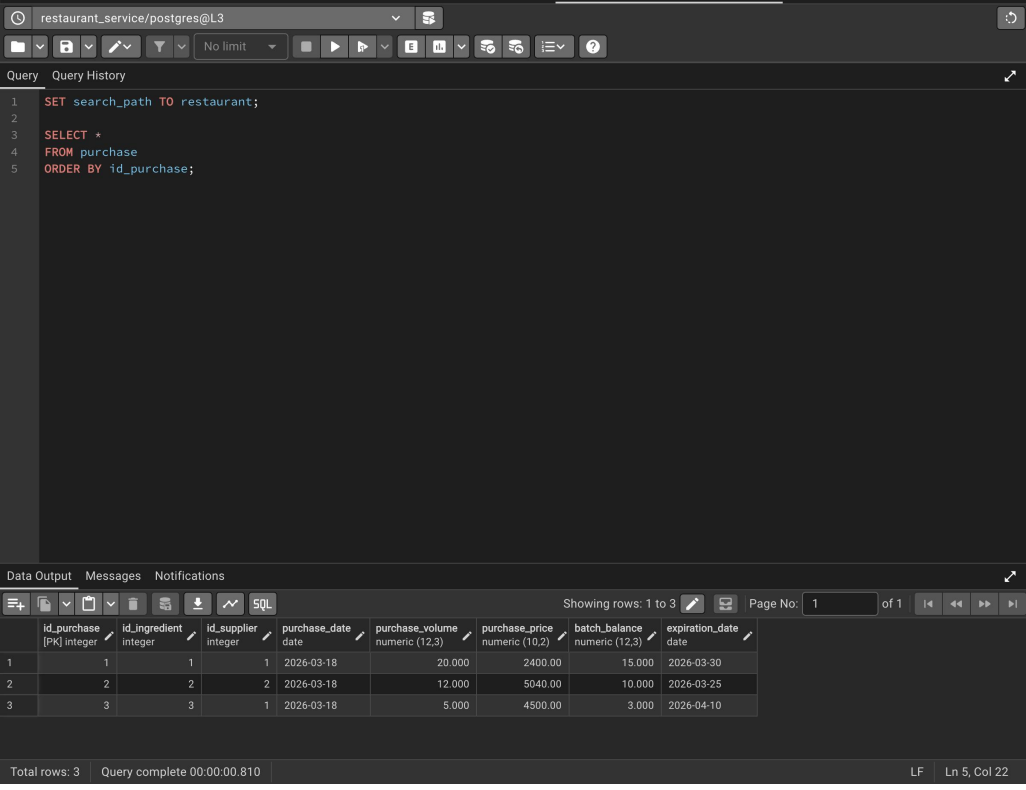
SET search_path TO restaurant;

SELECT *
FROM purchase
ORDER BY id_purchase;

DELETE FROM purchase
WHERE id_purchase = (
    SELECT p.id_purchase
    FROM purchase p
    ORDER BY p.batch_balance, p.id_purchase
    LIMIT 1
);

SELECT *
FROM purchase
ORDER BY id_purchase;

ROLLBACK;
```



The screenshot shows a PostgreSQL query editor interface. The query window displays a SQL script that sets the search path to 'restaurant', selects all records from the 'purchase' table ordered by 'id_purchase', deletes the record with the minimum 'batch_balance' (using a subquery), and then selects all records from 'purchase' again, ordered by 'id_purchase'. The results pane shows the state of the 'purchase' table after the deletion, with 3 rows remaining. The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.810'.

	id_purchase [PK] integer	id_ingredient integer	id_supplier integer	purchase_date date	purchase_volume numeric (12,3)	purchase_price numeric (10,2)	batch_balance numeric (12,3)	expiration_date date
1	1	1	1	2026-03-18	20.000	2400.00	15.000	2026-03-30
2	2	2	2	2026-03-18	12.000	5040.00	10.000	2026-03-25
3	3	3	1	2026-03-18	5.000	4500.00	3.000	2026-04-10

Total rows: 3 Query complete 00:00:00.810 LF Ln 5, Col 22

restaurant_service/postgres@L3

Query Query History

```
1 SET search_path TO restaurant;
2
3 DELETE FROM purchase
4 WHERE id_purchase = (
5     SELECT p.id_purchase
6     FROM purchase p
7     ORDER BY p.batch_balance, p.id_purchase
8     LIMIT 1
9 );
```

Data Output Messages Notifications

DELETE 1

Query returned successfully in 3 secs 207 msec.

✓ Query returned successfully in 3 secs 207 msec. ✕

Total rows: Query complete 00:00:03.207 LF Ln 9, Col 3

restaurant_service/postgres@L3

Query Query History

```
1 SET search_path TO restaurant;
2
3 SELECT *
4 FROM purchase
5 ORDER BY id_purchase;
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

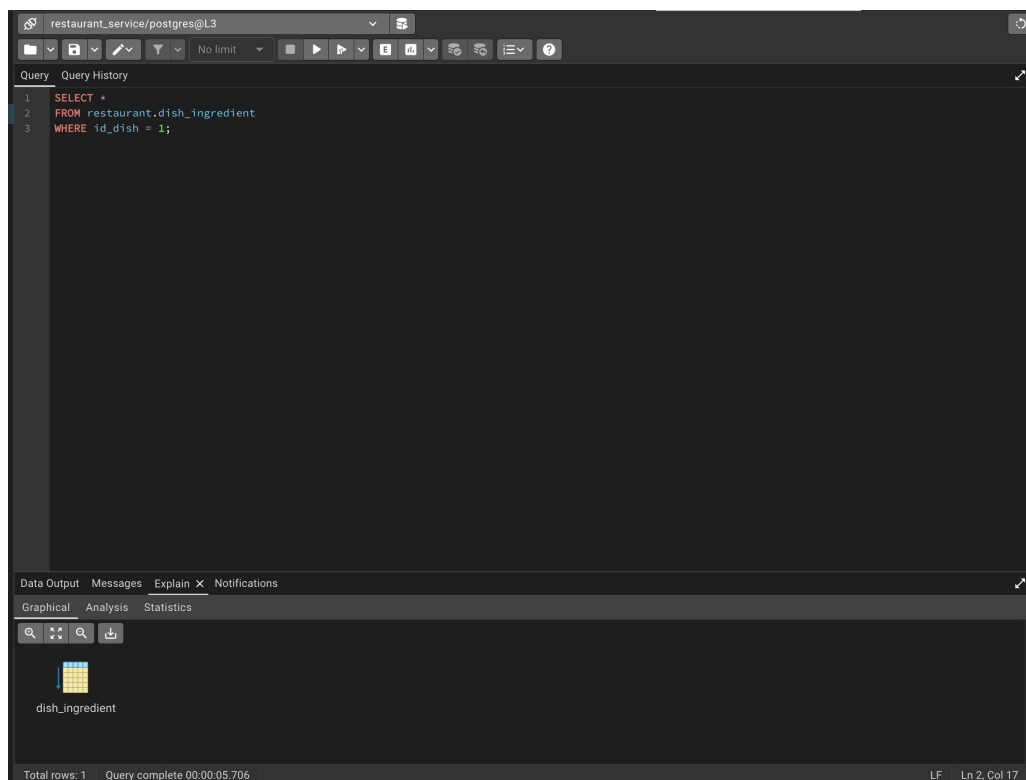
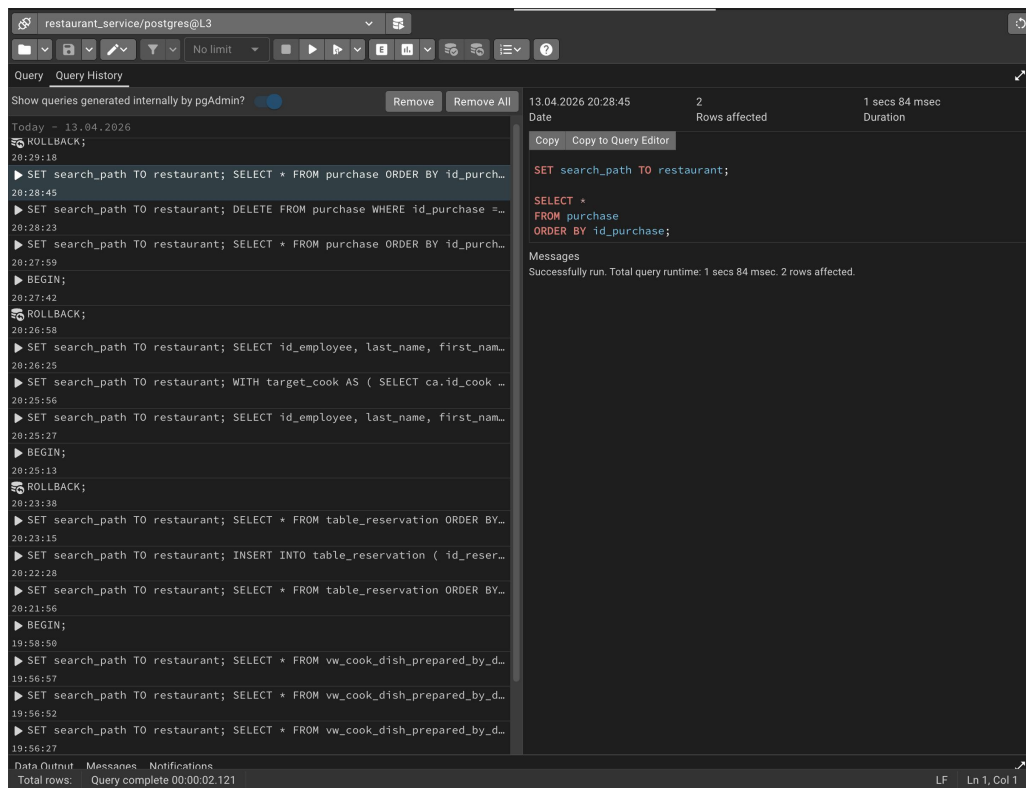
	id_purchase [PK] integer	id_ingredient integer	id_supplier integer	purchase_date date	purchase_volume numeric (12,3)	purchase_price numeric (10,2)	batch_balance numeric (12,3)	expiration_date date
1	1	1	1	2026-03-18	20.000	2400.00	15.000	2026-03-30
2	2	2	2	2026-03-18	12.000	5040.00	10.000	2026-03-25

✓ Successfully run. Total query runtime: 1 secs 84 msec. 2 rows affected. ✕

Total rows: 2 Query complete 00:00:01.084 LF Ln 5, Col 22

4.4. История запросов и графическое представление плана выполнения

В ходе работы был использован инструмент Query History для просмотра истории команд текущего сеанса, а также вкладка Explain для анализа графического представления плана выполнения запросов.



4.5. Создание индексов и сравнение времени выполнения запросов

В рамках лабораторной работы были исследованы два запроса:

- запрос к таблице `customer_order` с фильтрацией по официанту и периоду времени;
- запрос к таблице `dish_ingredient` с фильтрацией по полю `id_dish`.

Для первого запроса был создан составной индекс, а для второго — простой индекс.

Составной индекс

Листинг 17. Запрос к таблице `customer_order` без индекса

```
EXPLAIN ANALYZE
SELECT *
FROM restaurant.customer_order
WHERE id_waiter = 1
      AND order_datetime >= (
      SELECT MAX(order_datetime) - INTERVAL '30 days'
      FROM restaurant.customer_order
      );
```

The screenshot shows a PostgreSQL query editor with the following query:

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM restaurant.customer_order
4 WHERE id_waiter = 1
5      AND order_datetime >= (
6      SELECT MAX(order_datetime) - INTERVAL '30 days'
7      FROM restaurant.customer_order
8      );
```

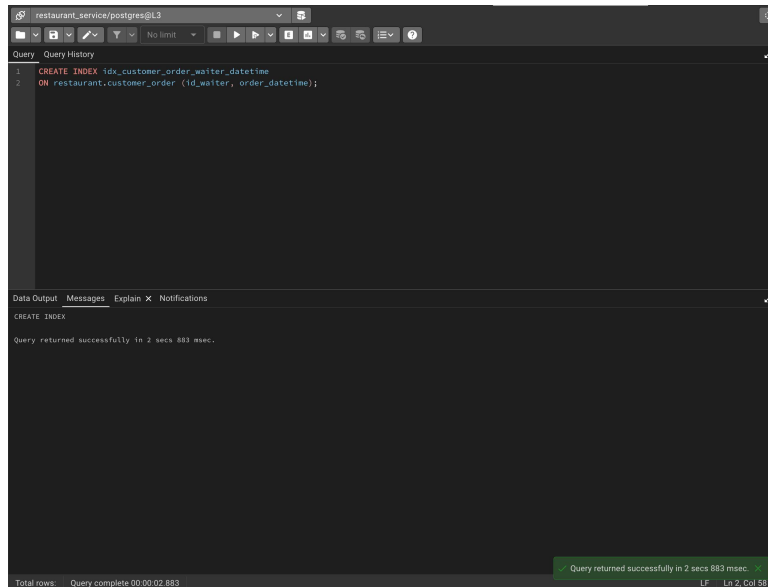
The execution plan is displayed below the query:

Step	Operation	Cost	Actual Time	Actual Rows	Actual Loops
1	Seq Scan on customer_order	(cost=11.39..23.04 rows=1 width=670)	(actual time=0.048..0.051 rows=2 loops=1)		
2	Filter: ((order_datetime >= (initPlan 1).col1) AND (id_waiter = 1))				
3	InitPlan 1				
4	-> Aggregate	(cost=11.38..11.39 rows=1 width=8)	(actual time=0.014..0.014 rows=1 loops=1)		
5	-> Seq Scan on customer_order customer_order_1	(cost=0.00..11.10 rows=110 width=8)	(actual time=0.004..0.005 rows=2 loops=1)		
6	Planning Time: 1.470 ms				
7	Execution Time: 0.114 ms				

Total rows: 7 Query complete 00:00:02.699

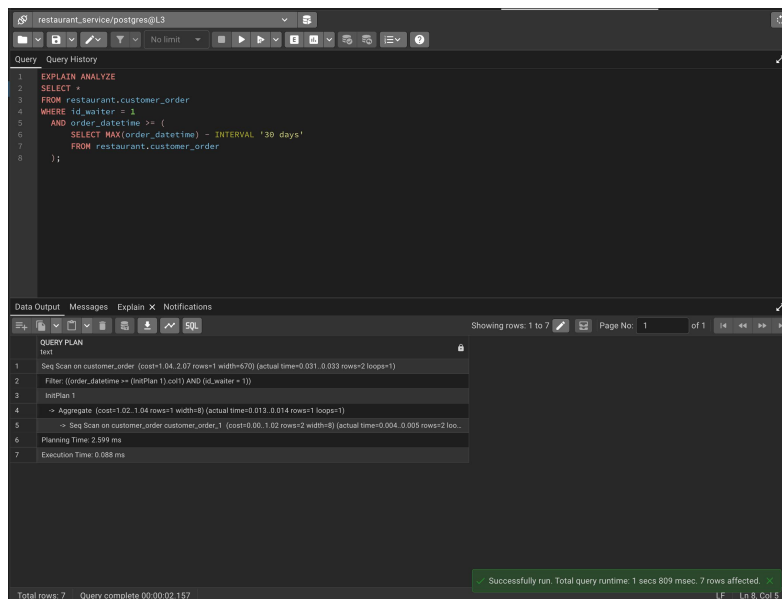
Листинг 18. Создание составного индекса для таблицы customer_order

```
CREATE INDEX idx_customer_order_waiter_datetime  
ON restaurant.customer_order (id_waiter, order_datetime);
```



Листинг 19. Запрос к таблице customer_order после создания составного индекса

```
EXPLAIN ANALYZE  
SELECT *  
FROM restaurant.customer_order  
WHERE id_waiter = 1  
AND order_datetime >= (  
    SELECT MAX(order_datetime) - INTERVAL '30 days'  
    FROM restaurant.customer_order  
);
```



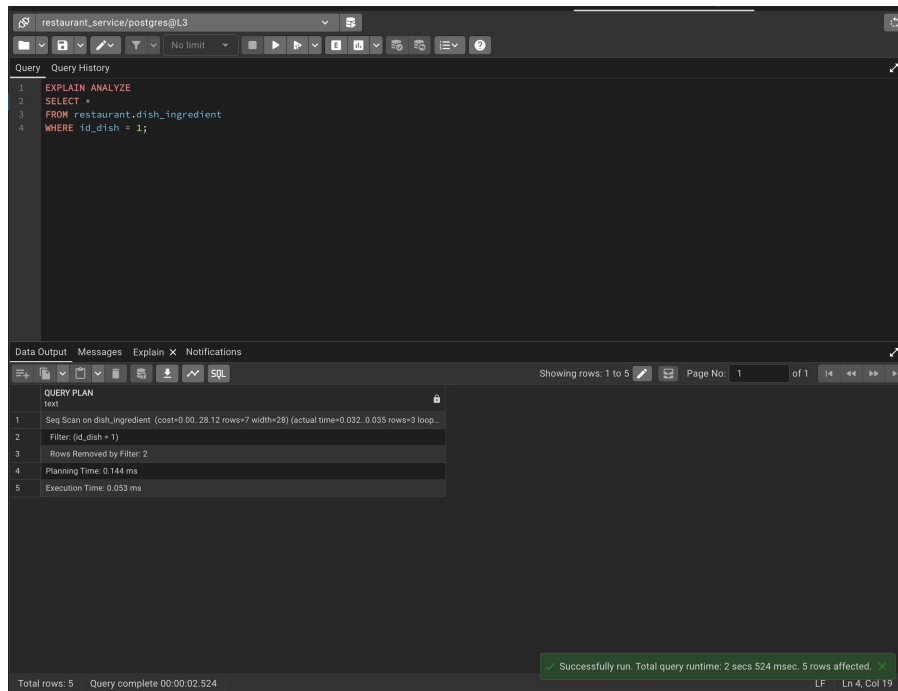
Листинг 20. Удаление составного индекса

```
DROP INDEX restaurant.idx_customer_order_waiter_datetime;
```

Простой индекс

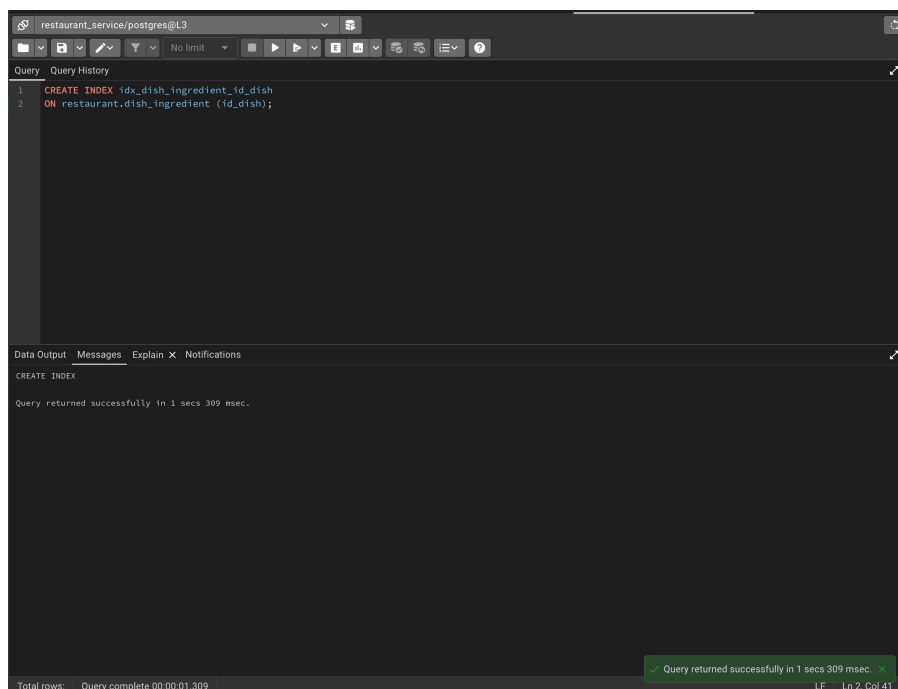
Листинг 21. Запрос к таблице dish_ingredient без индекса

```
EXPLAIN ANALYZE
SELECT *
FROM restaurant.dish_ingredient
WHERE id_dish = 1;
```



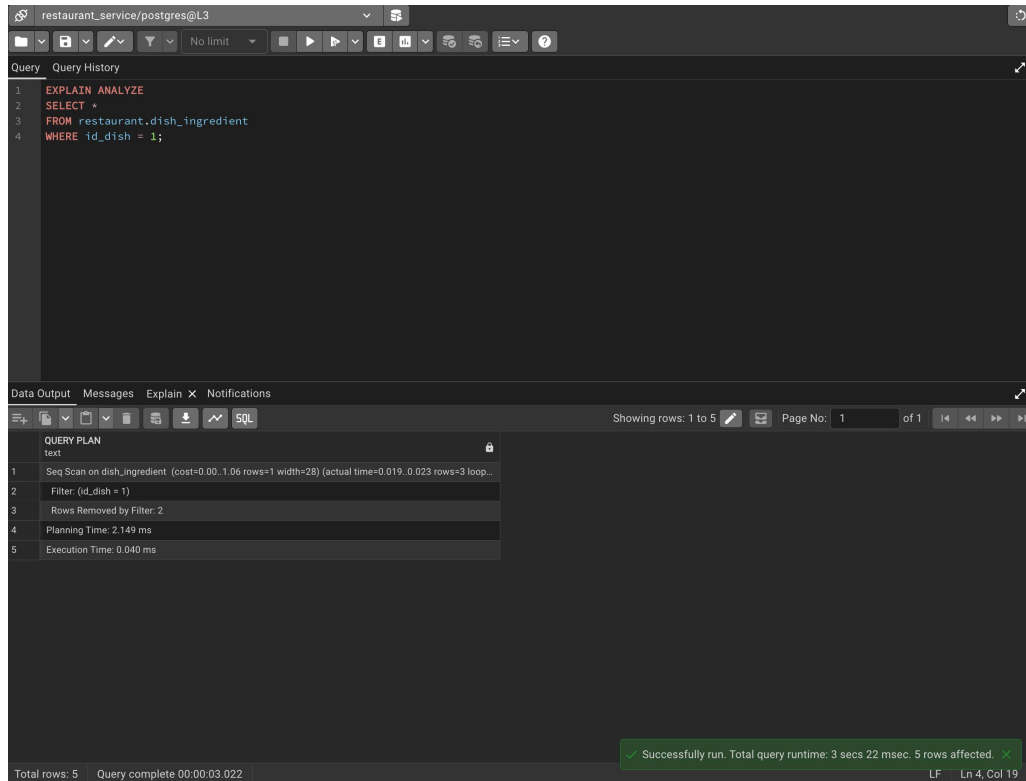
Листинг 22. Создание простого индекса для таблицы dish_ingredient

```
CREATE INDEX idx_dish_ingredient_id_dish
ON restaurant.dish_ingredient (id_dish);
```



Листинг 23. Запрос к таблице dish_ingredient после создания простого индекса

```
EXPLAIN ANALYZE
SELECT *
FROM restaurant.dish_ingredient
WHERE id_dish = 1;
```



Листинг 24. Удаление простого индекса

```
DROP INDEX restaurant.idx_dish_ingredient_id_dish;
```

Сравнение времени выполнения запросов

Результаты сравнения времени выполнения запросов представлены в таблице 1.

Таблица 1. Сравнение времени выполнения запросов без индекса и с индексом

Запрос	Без индекса	С индексом
Запрос к customer_order	0.114 ms	0.088 ms
Запрос к dish_ingredient	0.053 ms	0.040 ms

По результатам анализа видно, что в обоих случаях PostgreSQL продолжил использовать Seq Scan. Это объясняется тем, что объём тестовых данных в таблицах невелик, поэтому последовательное сканирование оказалось достаточно эффективным. При увеличении объёма данных использование индексов будет более значимым с точки зрения производительности.

5. Выводы

В ходе выполнения лабораторной работы были созданы запросы на выборку данных к базе данных PostgreSQL, построены представления, составлены запросы на модификацию данных с использованием подзапросов, изучены средства Query History и EXPLAIN, а также исследовано влияние индексов на время выполнения запросов.

В результате выполнения работы были приобретены практические навыки:

- составления запросов на выборку данных к базе данных PostgreSQL;
- создания и использования представлений;
- применения подзапросов в запросах на модификацию данных;
- анализа истории запросов и графического плана выполнения;
- создания простых и составных индексов;
- сравнительного анализа времени выполнения запросов без индексов и с индексами.

Таким образом, цель лабораторной работы достигнута: получены практические навыки работы с запросами на выборку и модификацию данных, представлениями и индексами в PostgreSQL.